

신규 온보딩 · 에이전트 · 스킬 사용법

malgn-agent, 이렇게 쓰세요

설치는 끝났고, 이제부터가 진짜입니다. PM 위임 모델의 핵심 개념부터 자주 쓰는 에이전트 10종, 실전 요청 시나리오, 자주 하는 실수까지 — 처음 쓰는 직원이 15분 안에 감을 잡도록 정리했습니다.

이 자료로 알게 되는 것

PM 위임 모델

자주 쓰는 에이전트 10종

실전 시나리오 4개

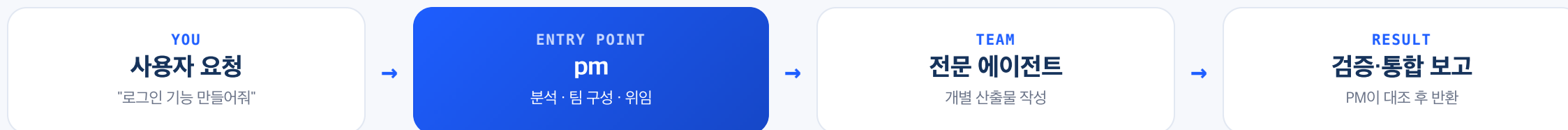
안티패턴

맑은소프트 전 직원 배포
설치 방법은 employee-guide 참고 · 이 문서는 "설치 이후" 편

당신은 에이전트를 지휘하지 않습니다

malgn-agent는 21개 전문 에이전트를 일일이 호출하는 도구모음이 아닙니다. 요청 하나를 PM에게 던지면, PM이 필요한 팀을 알아서 구성해 위임하고 검증까지 마친 뒤 결과만 돌려줍니다.

요청 하나가 처리되는 경로



기본값은 언제나 **pm(malgn-agent:pm)**입니다. 다른 에이전트를 직접 지정하지 않는 한, 모든 요청은 PM이 먼저 받아 분석하고 팀을 꾸립니다 — 어떤 전문가가 필요한지 직원이 미리 알 필요가 없습니다.

등록된 전문 에이전트

21

내부 참조 스킬

36

작업 판정 등급

5

직원이 외워야 할 것

0

직접 처리 vs 위임, 기준은 등급입니다

PM은 모든 요청을 5등급 중 하나로 판정합니다. Micro만 PM이 즉시 직접 처리하고, 나머지는 전문 에이전트에게 위임합니다 — 직원이 이 등급을 직접 매길 필요는 없습니다.

등급	기준	처리 방식
Micro	오탈자·조화·문구/CSS 한 줄 수정	PM이 즉시 직접 처리 (승인 불요)
Standard	일반 기능 개발·문서 작성·버그 수정	전문 에이전트에게 위임 + PM 단독 승인
Sensitive	DB·권한·인증·배포·결제·개인정보	위임 + reviewer 폴패널 + 사람 승인 필수
Exploration	원인 분석 중, 무엇을 고칠지 아직 모름	위임(시간·범위 제한) + 결론만 검증
Refactor	기능 변화 없는 구조 개선	위임 + reviewer 폴패널

한 줄 요약 — 애매하면 PM은 항상 무거운 등급으로 기웁니다. 직원은 등급을 계산하지 않아도, "이거 중요한 건인데?" 싶으면 그냥 요청만 하면 PM이 알아서 신중하게 다룹니다.

PM이 요청 하나를 처리하는 5단계

직원이 보는 것은 "요청 한 문장"과 "최종 보고" 둘뿐이지만, 그 사이에서 PM은 이 5단계를 거칩니다.

1

요청 분석

요청을 5등급(Micro~Refactor) 중 하나로 판정하고, 어떤 전문 영역이 필요한지 식별합니다.

2

팀 구성

업무 유형별 표준 패턴에 맞춰 에이전트 순서를 정합니다 (예: planner→architect→backend-dev→frontend-dev→qa-engineer).

3

위임

각 에이전트에게 입력 산출물·기대 결과·저장 경로를 명확히 전달하며 순차로 맡깁니다 (산출물 의존성 때문에 병렬이 아니라 순차).

4

검증

산출물이 실제로 존재하고 요구사항과 일치하는지 PM이 직접 대조합니다. 미흡하면 최대 2회 재작업을 지시합니다.

5

통합 보고

여러 에이전트의 산출물을 하나로 묶어 2~3분 안에 읽을 수 있는 결과로 요약해 전달합니다.

핵심 통찰

스킬 이름은 몰라도 됩니다

common-*, domain-* 스킬 36개는 에이전트가 작업 중 스스로 꺼내 쓰는 **내부 매뉴얼**입니다. 직원이 할 일은 스킬명을 외우거나 지정하는 게 아니라, **무엇을 원하는지 한 문장으로 명확히 말하는 것**뿐입니다.

"이 문서 슬라이드로 만들어줘"라고만 말하면, presenter가 스타일가이드 토큰을 어떻게 적용할지는 presenter의 몫입니다 — 직원은 결과물만 확인하면 됩니다.

기획 · 설계 — planner · architect

새로운 기능·서비스를 시작할 때 PM이 가장 먼저 배치하는 두 에이전트입니다.

pm · 항상 시작점

요청을 분석해 팀을 구성·위임하고, 결과를 검증해 통합 보고하는 프로젝트 매니저

언제: 별도로 다른 에이전트를 지정하지 않은 모든 요청

호출 예시: "로그인 기능 만들어줘" (그냥 이렇게만)

planner

고객 요구사항을 분석해 requirements.md·prd.md를 작성하는 기획 전문가

언제: 신규 기능·서비스를 기획 단계부터 시작할 때

호출 예시: "이 아이디어 기획서로 정리해줘"

architect

PRD 기반으로 시스템 아키텍처·기술스택·API 명세·데이터 모델을 설계하는 전문가

언제: 새 아키텍처 결정이 필요한 개발 착수 시점

호출 예시: "이 기획을 기술 설계로 만들어줘"

셋 다 **단독 호출도** 가능합니다 — 기획서만 필요하면 "planner"를 직접 지정해도 되고, 그냥 PM에게 요청하면 필요할 때 PM이 알아서 넣어줍니다.

구현 · 검증 — backend-dev · frontend-dev · qa-engineer

설계가 끝난 뒤, 실제로 동작하는 코드를 만들고 테스트하는 3단계입니다.

backend-dev

아키텍처 문서를 기반으로 실제 동작하는 백엔드 코드를 구현하는 전문가

언제: 설계 문서가 준비된 API·서버 로직 구현 단계

호출 예시: "이 설계대로 API 만들어줘"

frontend-dev

설계 문서를 기반으로 웹/앱 프론트엔드 UI를 구현하는 전문가

언제: 화면·UI 컴포넌트를 실제로 만들어야 할 때

호출 예시: "로그인 화면 만들어줘"

qa-engineer

구현된 코드를 테스트하고 결과를 보고하는 QA 전문가

언제: 백엔드·프론트 구현이 끝난 직후

호출 예시: "이 기능 테스트해줘"

세 에이전트는 **산출물 의존성이 있어 순차로만** 진행됩니다 — PM이 backend-dev 산출물을 frontend-dev에게 넘기고, 그 결과를 다시 qa-engineer에게 넘기는 식입니다.

검토 · 문서 · 발표 — reviewer · writer · presenter

결과물을 다듬고, 정식 문서로 만들고, 사람 앞에 내놓을 형태로 바꾸는 3단계입니다.

reviewer

리뷰 페르소나 패널을 구성해 산출물을 다관점으로 검토하고
통합 보고하는 진행자

언제: 중요 산출물을 최종 확정하기 전

호출 예시: "이 제안서/코드 검토해줘"

writer

제안서·보고서·기술 문서 등 비즈니스 문서를 전문적으로 작
성하는 전문가

언제: 정식 문서로 산출해야 할 때

호출 예시: "이 내용 보고서로 정리해줘"

presenter

문서 내용을 프레젠테이션 슬라이드로 구성해 PDF/HTML
로 만드는 전문가

언제: 발표 자료·슬라이드 매뉴얼이 필요할 때

호출 예시: "이 보고서 슬라이드로 만들어줘" [이 문서 자체가
예시](#)

지금 보고 있는 이 슬라이드 문서 자체가 **presenter**가 만든 산출물입니다 — writer/researcher가 만든 내용을 presenter가 재구성한 것이 아니라, 이번 경우는 실제 md/agent 파
일을 직접 근거로 삼아 작성했습니다.

에이전트 자체를 키우는 **trainer · evaluator**

지금까지의 8명과 결이 다릅니다 — 업무 결과물이 아니라 **malign-agent 플러그인 자체**를 개선하는 특수 목적 에이전트입니다.

trainer · 에이전트 교관

다른 에이전트들의 스킬을 분석하고 학습 자료를 정리해 knowledge-MD 초안을 작성

언제: 특정 에이전트를 더 잘 쓰게 만들고 싶을 때

호출 예시: "backend-dev 스킬업 시켜줘"

evaluator · 판정·승격 전담

에이전트/스킬/knowledge 산출물을 rubric으로 채점·판정하고 게이트 통과 시 PR로 승격까지 실행

언제: trainer 초안 완료 후, 또는 개선안을 실제 반영할 때

호출 예시: "이번 개선안 평가하고 승격해줘"

일반 업무 요청("~ 만들어줘")과 **에이전트 개선 요청**("~ 학습시켜줘")은 다른 트랙입니다. PM이 trainer 초안 → evaluator 판정·승격의 흐름으로 이어받으며, 사람 승인이 필요한 번경은 evaluator가 반드시 그 시점을 표시합니다.

스킬 36개, 직원은 몰라도 됩니다

스킬은 에이전트가 작업 중 스스로 참조하는 내부 노하우 문서입니다. 크게 세 갈래로 나뉩니다.

common-* · 9종

전 에이전트 공통 원칙

산출물 품질, 경로 관리, 검증 깊이, 정직한 보고 등 — 모든 에이전트가 상시 참조하는 기본기.

domain-* · 16종

특정 도메인 전문 지식

백엔드 보안, 시스템 설계, 제안 방법론, 테스트 기법 등 — 관련 에이전트만 필요할 때 꺼내 씀.

그 외 · 11종

운영 · 오케스트레이션

project-orchestration, agent-upskill, reflect-lessons 등 — PM·trainer·evaluator가 팀 운영·에이전트 성장에 사용.

직원이 할 일은 스킬명을 아는 게 아니라, **좋은 요청 문장**을 쓰는 것입니다 — "이거 검토해줘"라고만 해도 reviewer는 알아서 `reviewer-persona-panel-standard`를 참조합니다.

시나리오 1 · 신규 기능 개발 요청

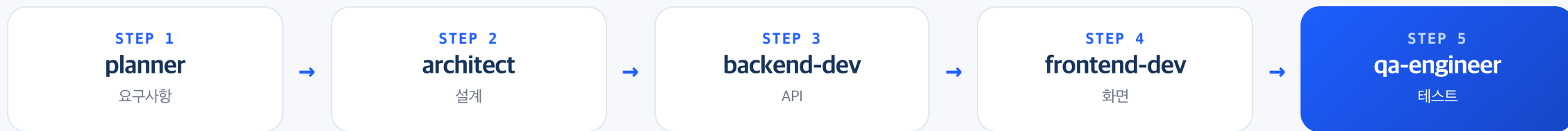
"로그인 기능 만들어줘" — 이 한 문장이 어떻게 5명의 팀으로 확장되는지 보여드립니다.

요청 문장

"로그인 기능 만들어줘"

PM 판정: **Standard 등급** — 여러 파일이 걸리지만 기능 범위가 명확 → 위임

PM이 구성하는 팀



각 단계는 **순차 진행**됩니다 — planner의 requirements.md를 architect가 읽고, architect의 설계서를 backend-dev/frontend-dev가 읽는 식입니다. PM은 매 단계 산출물을 검증한 뒤에만 다음으로 넘깁니다.

시나리오 2 · 산출물 검토 요청

"이 제안서 검토해줘" — reviewer는 혼자 읽고 의견을 내는 게 아니라, 관점이 다른 페르소나 패널을 직접 구성합니다.

요청 문장

"이 제안서 검토해줘"

PM 판정: 산출물 성격에 따라 **Standard(약식)** 또는 **Sensitive(폴패널)**

REVIEWER가 하는 일

1

패널 구성

프로젝트 성격에 맞는 페르소나 설계



2

다관점 리뷰

페르소나별로 각각 검토



3

통합 보고서

심각도(RAG) 분류해 PM에 전달

페르소나 패널에는 **발산형(비판적)** 관점 **최소 1명**이 항상 포함됩니다 — 좋은 말만 하는 패널이 아니라 진짜 결함을 찾는 것이 목적입니다.

시나리오 3 · 복합 요청 (조사+제안서)

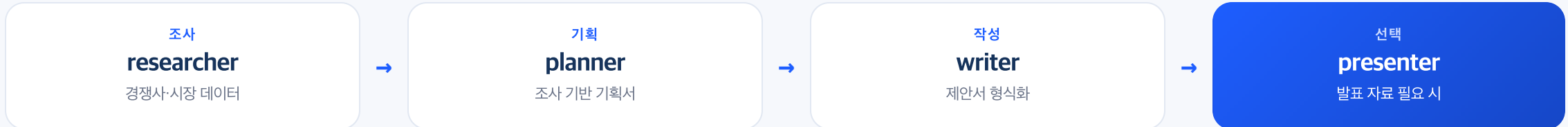
"경쟁사 분석 후 제안서 써줘" — 요청 하나에 두 가지 업무 유형이 섞여 있으면, PM이 먼저 분해합니다.

요청 문장

"경쟁사 분석 후 제안서 써줘"

PM 판정: 하위 작업 2그룹으로 분해 → **순차 배치**

그룹1 · 조사+기획 → 그룹2 · 문서화



복합 요청도 원칙은 같습니다 — **의존성이 있는 작업은 순차로만** 진행되고, PM이 각 단계 산출물을 다음 에이전트에게 넘기기 전에 검증합니다.

시나리오 4 · 발표 자료 제작 (지금 이 문서)

"이 문서를 슬라이드로 만들어줘" — 지금 보고 계신 이 매뉴얼 자체가 이 시나리오의 실제 결과물입니다.

이번 요청

"에이전트·스킬 사용법을 슬라이드 매뉴얼로 만들어줘"

PM 판정: **Standard 등급** — presenter 단독 위임

PRESENTER가 실제로 한 일

- **agents/*.md · skills/*/SKILL.md 실물 확인**
추측하지 않고 21개 에이전트, 36개 스킬 설명을 직접 grep해 근거로 삼음
- **html-스타일가이드-가로형.html 정보 그대로 적용**
CSS 토큰·클래스를 새로 만들지 않고 정보를 복사해 재사용
- **실제 로고 PNG 삽입 + grep 자기검수**
CSS 합성 로고 금지 규칙 준수, 토큰·클래스 드리프트 확인

자주 하는 실수 3가지

위임 모델에 익숙해지는 과정에서 가장 흔히 나오는 패턴입니다.

-  **에이전트 순서를 직접 지정 (마이크로매니징)**
"backend-dev 불러서 API 만들고 frontend-dev로 화면 만들고 qa-engineer로 테스트해줘" — PM의 역할을 대신하려는 패턴
-  **"무엇을" 원하는지만 말하기**
"로그인 기능 만들어줘"만 요청하면 팀 구성은 PM이 알아서 합니다
-  **사소한 수정까지 거창하게 위임 요청**
오타자 하나 고치는 데 "팀 꾸려서 검토까지 해줘" — Micro 등급은 PM이 즉시 처리하는 영역
-  **사소한 건 그냥 요청**
"이 문구 오타 고쳐줘"처럼 가볍게 요청하면 PM이 즉시 직접 처리합니다
-  **중요한 판단을 "급하니까" 직접 처리**
권한·결제·개인정보처럼 판단이 중요한 영역을 스스로 대충 고치는 경우 — Sensitive 등급은 반드시 위임+검증이 필요
-  **중요 영역은 급해도 PM을 통해 요청**
Sensitive 등급은 reviewer 풀패널 + 사람 승인까지 거치도록 설계돼 있습니다

PM이 못 하는 것도 있습니다

이 한계들은 버그가 아니라 설계입니다 — 중요한 결정에는 항상 사람 또는 전문 검증이 개입하도록 만든 안전장치입니다.

- **Sensitive/Refactor 등급은 PM 단독 승인 불가**

반드시 reviewer 풀패널을 거치고, 최종 승인은 사람(직원/대표)이 합니다

- **개별 산출물을 직접 작성하지 않음**

코드·설계·문서·디자인은 항상 전문 에이전트가 작성 — PM은 trivial 편집만 예외

- **전역 에이전트/스킬/knowledge 승격은 PM 권한 밖**

evaluator가 rubric으로 판정하고 git PR로 승격을 실행 — PM은 대상 선정만 담당

- **위임 실패가 2회 반복되면 PM이 직접 완수하지 않음**

담당 교체, 작업 재분해, 사람 에스컬레이션 중 하나를 택합니다 — 무한 재시도하지 않습니다

궁금하면 언제든 "이거 지금 어느 등급이야?", "누가 승인해야 해?"라고 PM에게 그냥 물어보세요 — 판단 기준을 숨기지 않습니다.

오늘 기억할 것 3가지

나머지는 다 잊어도 이 세 줄만 기억하면 됩니다.

01

기본은 그냥 PM에게 요청 — 팀 구성
은 알아서

02

애매하면 무거운 등급 — Sensitive는
반드시 검증+승인

03

스킬명은 몰라도 된다 — 좋은 한 문장
이면 충분

이 세 줄이 전부입니다. 21개 에이전트 이름, 36개 스킬명을 외울 필요는 처음부터 없었습니다.

첫 행동

지금, 한 문장으로 시작해보세요

malgn-agent가 설치돼 있다면 별도 지정 없이 그냥 요청하세요 — pm이 받아서 등급을 판정하고 팀을 꾸립니다.

가장 작은 첫 행동

"이번 주 정리한 자료, 슬라이드로 만들어줘" 처럼 지금 하고 있는 작업 하나를 그대로 던져보세요.

문의

설치 방법은 [employee-guide.html](#) 참고 · 궁금한 점은 dev@malgnsoft.com

malgn-agent v1.5.0
맑은소프트 전 직원 배포